UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course Title: Software Engineering Lab | Course Code: CSE 314 | Section: B2



REST API DOCUMENTATION & INTEGRATION TEST PLAN

SerialSync

Implemented REST & WebSocket Contracts for Multi-Location Booking, Live Queue Tracking, Home Diagnostics, and Platform Moderation [MERN Stack]

SUBMITTED BY:	SUBMITTED TO:
FM Maftahul Jannat (23201084) · Project Manager Md Adil Hossain (23201102) · Team Lead Md Abu Rafe Mostak Hossain Fahim (23101059) · QA Lead Rohan Hasan Khan (23201108) · Report Lead	**Md. Ashraful Alam** Lecturer Department of Computer Science & Engineering University of Asia Pacific (UAP)

Document Date: 29 October 2026 | Academic Session: Fall 2026

Companion to Software Requirements Specification (SRS) Chapters 6 and 7.
Base URL: http://localhost:5050 · 162 automated tests (66 pass-path, 96 fail-path).

6. API and Testing Documentation

SerialSync exposes a RESTful API together with a bi-directional WebSocket interface (Socket.io) over HTTP(S) and WSS. All REST endpoints consume and produce JSON (application/json) except multipart report upload and PDF download. Standard HTTP status codes are used: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, and 500 Internal Server Error.

This chapter documents every endpoint (Sections 6.1 to 6.7) and the automated pass and fail test cases that verify them (Sections 6.8 to 6.11). Paths below are the implemented Express 5 routes under /api/* (not /api/v1/*). Successful responses use { success: true, ... }. Errors use { success: false, message: "..." }. Session is an HTTP-only cookie named serialsync_token (7-day JWT) or Authorization: Bearer <token>.

Conventions

HTTP code	Meaning
200 OK	Successful GET / PATCH / POST mutation without a new resource
201 Created	Resource created (register, chamber, schedule, booking, walk-in, order)
400 Bad Request	Validation failure (missing fields, invalid enum, weak password)
401 Unauthorized	Missing or invalid session cookie / Bearer token
403 Forbidden	Role or doctor-approval gate (DOCTOR_PENDING)
404 Not Found	Unknown id
409 Conflict	Duplicate booking or overlapping schedule

Role	Typical access
patient	Book serials, live queue, diagnostic orders, profile
doctor	Chambers, schedules, day itinerary (after admin approval)
assistant	Queue console: advance, walk-in, delay, pause
phlebotomist	Assigned diagnostic dispatch queue and status
admin	Doctor BMDC approval, reports upload, all privileged routes

6.1 Module 1: Authentication & User Management

GET /api/health

Access: Public

Description: Service liveness probe used by deployment health checks.

Parameter	Type	Required	Description
?	?	No	No parameters.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "service": "serialsync-api",
  "status": "ok"
}
```

POST /api/auth/register

Access: Public (All Users)

Description: Registers a new account. Password is hashed with bcrypt (12 salt rounds). Doctors are created with doctorApprovalStatus = pending. Sets the serialsync_token cookie.

Parameter	Type	Required	Description
fullName	String	Yes	Full legal name of the user.

email	String	Yes	Unique email used for login and notifications.
password	String	Yes	Minimum 8 characters.
phone	String	No	Bangladeshi mobile number (01XXXXXXXX).
role	Enum	No	patient (default), doctor, assistant, phlebotomist, admin (admin blocked in production).
doctorProfile	Object	No	Optional specialtyTitle, specialtySlug, affiliations (doctors).

Sample Request Body

```
{
  "fullName": "Md Adil Hossain",
  "email": "adil@example.com",
  "phone": "01712345678",
  "password": "SecurePassword123",
  "role": "patient"
}
```

Sample JSON Response (201)

```
{
  "success": true,
  "user": {
    "id": "68f1a2b3c4d5e6f708091011",
    "fullName": "Md Adil Hossain",
    "email": "adil@example.com",
    "phone": "01712345678",
    "role": "patient"
  }
}
```

POST /api/auth/login

Access: Public (All Users)

Description: Authenticates with email and password; returns the user and sets the HTTP-only JWT cookie.

Parameter	Type	Required	Description
email	String	Yes	Registered account email.
password	String	Yes	Account plaintext password.

Sample Request Body

```
{
  "email": "adil@example.com",
  "password": "SecurePassword123"
}
```

Sample JSON Response (200)

```
{
  "success": true,
  "user": {
    "id": "68f1a2b3c4d5e6f708091011",
    "fullName": "Md Adil Hossain",
    "email": "adil@example.com",
    "role": "patient"
  }
}
```

POST /api/auth/logout

Access: Authenticated User

Description: Clears the session cookie. Subsequent protected requests return 401.

Parameter	Type	Required	Description
Cookie	Header	Yes	serialsync_token session cookie.

Sample Request Body

```
None (POST with Cookie)
```

Sample JSON Response

```
{
  "success": true,
  "message": "Logged out"
}
```

GET /api/auth/me

Access: Authenticated User

Description: Returns the currently authenticated user from the JWT session.

Parameter	Type	Required	Description
Authorization	Header	Yes	Cookie serialsync_token or Bearer token.

Sample Request Body

```
None (GET Request with Cookie/Header)
```

Sample JSON Response

```
{
  "success": true,
  "user": {
    "id": "68f1a2b3c4d5e6f708091011",
    "fullName": "Md Adil Hossain",
    "role": "patient"
  }
}
```

POST /api/auth/forgot-password

Access: Public

Description: Starts password reset. In non-production, the response includes resetToken for QA.

Parameter	Type	Required	Description
email	String	Yes	Registered email address.

Sample Request Body

```
{
  "email": "adil@example.com"
}
```

Sample JSON Response

```
{
  "success": true,
  "message": "If that email exists, a reset link was sent",
  "resetToken": "a1b2c3d4..."
}
```

POST /api/auth/reset-password

Access: Public

Description: Completes password reset using the token from forgot-password.

Parameter	Type	Required	Description
token	String	Yes	One-time reset token.
password	String	Yes	New password (min 8 characters).

Sample Request Body

```
{
  "token": "a1b2c3d4...",
  "password": "newpassword99"
}
```

Sample JSON Response

```
{
  "success": true,
  "message": "Password updated"
}
```

POST /api/auth/change-password

Access: Authenticated User

Description: Changes password for the logged-in user after verifying the current password.

Parameter	Type	Required	Description
currentPassword	String	Yes	Existing password.
newPassword	String	Yes	New password (min 8 characters).

Sample Request Body

```
{
  "currentPassword": "SecurePassword123",
  "newPassword": "EvenSaferPass99"
}
```

Sample JSON Response

```
{
  "success": true,
  "message": "Password changed"
}
```

GET /api/users/me

Access: Authenticated User

Description: Retrieves the logged-in profile including patientProfile or doctorProfile.

Parameter	Type	Required	Description

Authorization	Header	Yes	Session cookie or Bearer token.

Sample Request Body

```
None (GET Request with Cookie/Header)
```

Sample JSON Response

```
{
  "success": true,
  "user": {
    "id": "68f1a2b3c4d5e6f708091011",
    "fullName": "Md Adil Hossain",
    "email": "adil@example.com",
    "phone": "01712345678",
    "role": "patient",
    "patientProfile": {
      "city": "Dhaka",
      "bloodType": "B+",
      "smartPassVerified": true
    }
  }
}
```

PATCH /api/users/me

Access: Authenticated User

Description: Updates fullName, phone, and role-specific profile fields.

Parameter	Type	Required	Description
fullName	String	No	Updated display name.
phone	String	No	Updated mobile number.
patientProfile	Object	No	city, bloodType, smartPassVerified (patients).
doctorProfile	Object	No	specialtyTitle, affiliations, roomLabel (doctors).

Sample Request Body

```
{
  "fullName": "Updated Name",
  "phone": "01811111111",
  "patientProfile": { "city": "Dhaka" }
}
```

Sample JSON Response

```
{
  "success": true,
  "user": {
    "id": "68f1a2b3c4d5e6f708091011",
    "fullName": "Updated Name",
    "phone": "01811111111",
    "role": "patient"
  }
}
```

GET /api/users/admin/overview

Access: System Administrator

Description: RBAC smoke endpoint confirming admin session.

Parameter	Type	Required	Description
Authorization	Header	Yes	Admin session.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "message": "Admin RBAC route",
  "admin": { "id": "...", "role": "admin" }
}
```

6.2 Module 2: Doctor Multi-Location Schedules

GET /api/doctors

Access: Authenticated User

Description: Searches the public directory of approved doctors by specialty, city, and free-text query.

Parameter	Type	Required	Description

q	Query	No	Name / affiliation search string.
specialty	Query	No	Specialty slug (e.g. cardiology).
city	Query	No	City filter (e.g. Dhaka).

Sample Request Body

```
None (Query Parameters)
```

Sample JSON Response

```
{
  "success": true,
  "doctors": [{
    "id": "68f1doc...",
    "fullName": "Prof. Dr. M. Rahman",
    "doctorProfile": {
      "specialtyTitle": "Cardiologist",
      "affiliations": "Green Life Hospital"
    }
  }],
  "meta": { "specialties": ["cardiology", "medicine"] }
}
```

GET /api/doctors/:doctorId

Access: Authenticated User

Description: Returns a doctor profile with chambers and active weekly schedules for the booking UI.

Parameter	Type	Required	Description
doctorId	Path	Yes	Doctor User ObjectId.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "doctor": {
    "id": "68f1doc...",
    "fullName": "Prof. Dr. M. Rahman",
    "doctorApprovalStatus": "approved"
  },
  "chambers": [{ "id": "ch_01", "name": "Green Life", "area": "Dhanmon
  "schedules": [{ "id": "sch_01", "daysOfWeek": [1, 3], "startTime": "
}
```

POST /api/chambers

Access: Verified Doctor (approved)

Description: Creates a hospital or private clinic chamber owned by the authenticated doctor. Pending doctors receive 403 DOCTOR_PENDING.

Parameter	Type	Required	Description
name	String	Yes	Chamber / clinic display name.
address	String	No	Street address.
city	String	No	City (default Dhaka).
area	String	No	Area (e.g. Dhanmondi).
phone	String	No	Chamber contact number.

Sample Request Body

```
{
  "name": "Green Life Hospital",
  "address": "32 Green Road",
  "city": "Dhaka",
  "area": "Dhanmondi"
}
```

Sample JSON Response (201)

```
{
  "success": true,
  "chamber": {
    "id": "68f1ch...",
    "name": "Green Life Hospital",
    "area": "Dhanmondi",
    "isActive": true
  }
}
```

GET /api/chambers

Access: Verified Doctor (approved)

Description: Lists chambers owned by the authenticated doctor.

Parameter	Type	Required	Description
Authorization	Header	Yes	Doctor session.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "chambers": [{ "id": "68f1ch...", "name": "Green Life Hospital" }]
}
```

PATCH /api/chambers/:id

Access: Verified Doctor (approved)

Description: Updates chamber metadata (name, address, area, phone, isActive).

Parameter	Type	Required	Description
id	Path	Yes	Chamber ObjectId.
name	String	No	Updated facility name.
isActive	Boolean	No	Soft-disable chamber.

Sample Request Body

```
{
  "phone": "01711112222",
  "isActive": true
}
```

Sample JSON Response

```
{
  "success": true,
  "chamber": { "id": "68f1ch...", "phone": "01711112222" }
}
```

GET /api/chambers/directory

Access: Queue Assistant, Administrator

Description: Cross-doctor chamber directory for assistant console session setup.

Parameter	Type	Required	Description
Authorization	Header	Yes	Assistant or admin session.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "chambers": [{
    "id": "68f1ch...",
    "name": "Green Life Hospital",
    "doctorId": "68f1doc..."
  }]
}
```

POST /api/schedules

Access: Verified Doctor (approved)

Description: Creates a recurring weekly availability at a chamber with overlap detection (409 on conflict).

Parameter	Type	Required	Description
chamberId	String	Yes	Owned Chamber ObjectId.
daysOfWeek	Integer[]	Yes	0=Sun ? 6=Sat.
startTime	String	Yes	24h start (HH:MM).
endTime	String	Yes	24h end (HH:MM), after startTime.
consultationFee	Number	Yes	Fee in BDT.

slotDurationMinutes	Integer	Yes	Minutes per serial (5?120).
patientsPerSlot	Integer	No	Quota per slot (default 1).
notes	String	No	Optional chamber notes.

Sample Request Body

```
{
  "chamberId": "68f1ch...",
  "daysOfWeek": [1, 3],
  "startTime": "18:00",
  "endTime": "21:00",
  "consultationFee": 1200,
  "patientsPerSlot": 2,
  "slotDurationMinutes": 15
}
```

Sample JSON Response (201)

```
{
  "success": true,
  "schedule": {
    "id": "68f1sch...",
    "doctorId": "68f1doc...",
    "daysOfWeek": [1, 3],
    "startTime": "18:00",
    "endTime": "21:00",
    "status": "active"
  }
}
```

GET /api/schedules/me

Access: Verified Doctor (approved)

Description: Lists the doctor's own schedules including inactive rows.

Parameter	Type	Required	Description
Authorization	Header	Yes	Doctor session.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "schedules": [{ "id": "68f1sch...", "status": "active" }]
}
```

GET /api/schedules/doctors/:doctorId

Access: Public

Description: Public active schedules and chambers for a doctor. Inactive schedules are omitted.

Parameter	Type	Required	Description
doctorId	Path	Yes	Doctor User ObjectId.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "schedules": [{ "id": "68f1sch...", "daysOfWeek": [1, 3] }],
  "chambers": [{ "id": "68f1ch...", "name": "Green Life Hospital" }]
}
```

GET /api/schedules/doctors/:doctorId/slots

Access: Public

Description: Expands weekly schedules into bookable slots for a date range, with remaining quota per slot.

Parameter	Type	Required	Description
doctorId	Path	Yes	Doctor User ObjectId.
from	Query	Yes	Start date YYYY-MM-DD.
to	Query	Yes	End date YYYY-MM-DD.

Sample Request Body

```
None (Query Parameters)
```

Sample JSON Response

```
{
  "success": true,
  "slots": [{
    "scheduleId": "68f1sch...",
    "date": "2026-10-05",
    "slotStart": "18:00",
    "slotEnd": "18:15",
    "remaining": 2
  }]
}
```

PATCH /api/schedules/:id

Access: Verified Doctor (approved)

Description: Updates times, days, fee, or chamber. Overlaps return 409.

Parameter	Type	Required	Description
id	Path	Yes	Schedule ObjectId.
startTime	String	No	Updated 24h start.
endTime	String	No	Updated 24h end.
daysOfWeek	Integer[]	No	Updated weekday set.
consultationFee	Number	No	Updated fee in BDT.

Sample Request Body

```
{
  "startTime": "18:30",
  "endTime": "21:30",
  "consultationFee": 1500
}
```

Sample JSON Response

```
{
  "success": true,
  "schedule": {
    "id": "68f1sch...",
    "startTime": "18:30",
    "consultationFee": 1500
  }
}
```

PATCH /api/schedules/:id/status

Access: Verified Doctor (approved)

Description: Toggles schedule visibility. Inactive schedules disappear from the public booking view.

Parameter	Type	Required	Description
id	Path	Yes	Schedule ObjectId.
status	Enum	Yes	active or inactive.

Sample Request Body

```
{
  "status": "inactive"
}
```

Sample JSON Response

```
{
  "success": true,
  "schedule": { "id": "68f1sch...", "status": "inactive" }
}
```

DELETE /api/schedules/:id

Access: Verified Doctor (approved)

Description: Deletes an availability row owned by the doctor.

Parameter	Type	Required	Description
id	Path	Yes	Schedule ObjectId.

Sample Request Body

```
None (DELETE Request)
```

Sample JSON Response

```
{
  "success": true,
  "message": "Schedule deleted"
}
```

6.3 Module 3: Appointment Booking & Dynamic Serial Allocation

POST /api/schedules/slots/book

Access: Authenticated Patient, Administrator

Description: Atomically reserves a sequential serial for the doctor-day (Redis INCR with in-memory fallback) and creates a SlotBooking with confirmationCode.

Parameter	Type	Required	Description
scheduleId	String	Yes	Target DoctorSchedule ObjectId.
date	String	Yes	Appointment date YYYY-MM-DD.
slotStart	String	Yes	Slot start time HH:MM from the slots API.
patientNote	String	No	Optional note for the doctor.

Sample Request Body

```
{
  "scheduleId": "68f1sch...",
  "date": "2026-10-05",
  "slotStart": "18:00"
}
```

Sample JSON Response (201)

```
{
  "success": true,
  "appointment": {
    "id": "68f1apt...",
    "serialNumber": 14,
    "confirmationCode": "SS-A1B2C3",
    "status": "booked",
    "date": "2026-10-05",
    "slotStart": "18:00"
  }
}
```

GET /api/appointments/me

Access: Authenticated Patient, Administrator

Description: Chronological list of the patient's bookings (optional status filter).

Parameter	Type	Required	Description
status	Query	No	booked, confirmed, completed, cancelled, no_show.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "appointments": [{
    "id": "68f1apt...",
    "serialNumber": 14,
    "status": "booked",
    "doctor": { "fullName": "Prof. Dr. M. Rahman" },
    "chamber": { "name": "Green Life Hospital" }
  }]
}
```

GET /api/appointments/doctor/day

Access: Verified Doctor, Administrator

Description: Doctor chamber itinerary for a calendar day.

Parameter	Type	Required	Description
date	Query	No	YYYY-MM-DD (defaults to today).

Sample Request Body

```
None (Query Parameters)
```

Sample JSON Response

```
{
  "success": true,
  "date": "2026-10-05",
  "appointments": [{
    "serialNumber": 1,
    "slotStart": "18:00",
    "status": "booked"
  }]
}
```

GET /api/appointments/:id

Access: Owner Patient, Assigned Doctor, Administrator

Description: Appointment detail. Other patients receive 403.

Parameter	Type	Required	Description
id	Path	Yes	SlotBooking ObjectId.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "appointment": {
    "id": "68f1apt...",
    "serialNumber": 14,
    "confirmationCode": "SS-A1B2C3",
    "status": "booked"
  }
}
```

GET /api/appointments/:id/qr

Access: Owner Patient, Administrator

Description: Returns a PNG data URL QR encoding confirmationCode, serialNumber, and chamber metadata (SmartPass).

Parameter	Type	Required	Description
id	Path	Yes	SlotBooking ObjectId.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "qrDataUrl": "...",
  "payload": {
    "appointmentId": "68f1apt...",
    "confirmationCode": "SS-A1B2C3",
    "serialNumber": 14,
    "date": "2026-10-05"
  }
}
```

POST /api/appointments/:id/cancel

Access: Owner Patient, Administrator

Description: Cancels a booked/confirmed appointment. Completed visits cannot be cancelled (400).

Parameter	Type	Required	Description
id	Path	Yes	SlotBooking ObjectId.

Sample Request Body

```
None (POST Request)
```

Sample JSON Response

```
{
  "success": true,
  "appointment": {
    "id": "68f1apt...",
    "status": "cancelled"
  }
}
```

PATCH /api/appointments/:id/status

Access: Assigned Doctor, Administrator

Description: Doctor updates visit workflow status with allowed transitions.

Parameter	Type	Required	Description
id	Path	Yes	SlotBooking ObjectId.
status	Enum	Yes	confirmed, completed, no_show, cancelled.

Sample Request Body

```
{
  "status": "completed"
}
```

Sample JSON Response

```
{
  "success": true,
  "appointment": { "id": "68f1apt...", "status": "completed" }
}
```

6.4 Module 4: Queue Assistant & Chamber Operations

Queue mutations emit Socket.io event queue_updated to room queue:<sessionId>. Clients connect to path /socket.io with the same JWT cookie (or handshake auth.token), then emit join_queue { sessionId }.

POST /api/queue/sessions

Access: Queue Assistant, Doctor, Administrator

Description: Ensures a QueueSession for doctor + chamber + date and returns the built queue state.

Parameter	Type	Required	Description
doctorId	String	Yes	Doctor User ObjectId.
chamberId	String	Yes	Chamber ObjectId.
date	String	Yes	Session date YYYY-MM-DD.

Sample Request Body

```
{
  "doctorId": "68f1doc...",
  "chamberId": "68f1ch...",
  "date": "2026-10-05"
}
```

Sample JSON Response

```
{
  "success": true,
  "queue": {
    "sessionId": "68f1qs...",
    "date": "2026-10-05",
    "status": "open",
    "servingIndex": -1,
    "queueLength": 12,
    "nowServing": null,
    "items": [{ "label": "1", "kind": "booking" }]
  }
}
```

GET /api/queue/sessions/:id

Access: Authenticated User

Description: Full queue snapshot: nowServing, upNext, delay, pause, and item list.

Parameter	Type	Required	Description
id	Path	Yes	QueueSession ObjectId.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "queue": {
    "sessionId": "68f1qs...",
    "nowServing": { "label": "11", "kind": "booking" },
    "upNext": [{ "label": "12" }],
    "delayMinutes": 0
  }
}
```

GET /api/queue/me/today

Access: Authenticated Patient

Description: Live position for the patient's appointment on the current local date, including patientsAhead and ETA.

Parameter	Type	Required	Description
Authorization	Header	Yes	Patient session.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "appointmentId": "68f1apt...",
  "serialNumber": 14,
  "patientsAhead": 3,
  "estimatedWaitMinutes": 45,
  "queue": { "sessionId": "68f1qs...", "nowServing": { "label": "11" }
}
```

POST /api/queue/sessions/:id/advance

Access: Queue Assistant, Administrator

Description: Advances servingIndex by +1 and emits queue_updated to waiting patients.

Parameter	Type	Required	Description
id	Path	Yes	QueueSession ObjectId.

Sample Request Body

```
None (POST Request)
```

Sample JSON Response

```
{
  "success": true,
  "queue": {
    "sessionId": "68f1qs...",
    "servingIndex": 0,
    "nowServing": { "label": "1", "kind": "booking" }
  }
}
```

POST /api/queue/sessions/:id/walk-in

Access: Queue Assistant, Administrator

Description: Injects a walk-in with sub-index (e.g. 11B) after afterSerial without renumbering later bookings.

Parameter	Type	Required	Description
id	Path	Yes	QueueSession ObjectId.
afterSerial	Integer	Yes	Insert after this booked serial.
patientName	String	Yes	Walk-in display name.
priority	Enum	No	emergency | urgent | standard.
reason	String	No	Optional clinical reason.

Sample Request Body

```
{
  "afterSerial": 11,
  "patientName": "Rahim Uddin",
  "priority": "emergency"
}
```

Sample JSON Response (201)

```
{
  "success": true,
  "walkIn": {
    "displayLabel": "11B",
    "patientName": "Rahim Uddin"
  },
  "queue": { "queueLength": 13 }
}
```

POST /api/queue/sessions/:id/no-show

Access: Queue Assistant, Administrator

Description: Marks a booked patient as no-show and skips them in the serving order.

Parameter	Type	Required	Description
id	Path	Yes	QueueSession ObjectId.
bookingId	String	Yes	SlotBooking ObjectId.

Sample Request Body

```
{
  "bookingId": "68f1apt..."
}
```

Sample JSON Response

```
{
  "success": true,
  "queue": { "sessionId": "68f1qs..." }
}
```

POST /api/queue/sessions/:id/undo

Access: Queue Assistant, Administrator

Description: Reverses the last no-show action on the session.

Parameter	Type	Required	Description
id	Path	Yes	QueueSession ObjectId.

Sample Request Body

```
None (POST Request)
```

Sample JSON Response

```
{
  "success": true,
  "queue": { "sessionId": "68f1qs..." }
}
```

POST /api/queue/sessions/:id/delay

Access: Queue Assistant, Administrator

Description: Sets delayMinutes / delayReason; clients recompute ETA from queue_updated.

Parameter	Type	Required	Description
id	Path	Yes	QueueSession ObjectId.
delayMinutes	Integer	Yes	Estimated extra wait (e.g. 15, 30, 45).
delayReason	String	No	e.g. Traffic on Mirpur Road.

Sample Request Body

```
{
  "delayMinutes": 30,
  "delayReason": "Traffic on Mirpur Road"
}
```

Sample JSON Response

```
{
  "success": true,
  "queue": {
    "delayMinutes": 30,
    "delayReason": "Traffic on Mirpur Road"
  }
}
```

POST /api/queue/sessions/:id/pause

Access: Queue Assistant, Administrator

Description:

Pauses or resumes the chamber session with an optional public message.

Parameter	Type	Required	Description
id	Path	Yes	QueueSession ObjectId.
paused	Boolean	Yes	true to pause, false to resume.
pauseMessage	String	No	Message shown on the live tracker.

Sample Request Body

```
{
  "paused": true,
  "pauseMessage": "Doctor in emergency OT"
}
```

Sample JSON Response

```
{
  "success": true,
  "queue": {
    "status": "paused",
    "pauseMessage": "Doctor in emergency OT"
  }
}
```

6.4.1 WebSocket contract (Socket.io)

Event	Direction	Payload	Description
join_queue	Client ? Server	{ sessionId }	Subscribe to room queue:<sessionId>.
leave_queue	Client ? Server	{ sessionId }	Unsubscribe from the room.
queue_updated	Server ? Client	Queue state object	Emitted after REST queue mutations.

Handshake authentication: cookie serialsync_token or socket.handshake.auth.token. Path: /socket.io. CORS origin: CLIENT_URL.

6.5 Module 5: Home Diagnostics & Reports

GET **/api/diagnostics/tests**

Access: Authenticated User

Description: Active diagnostic test catalog (CBC, lipid, HbA1c, etc.).

Parameter	Type	Required	Description
search	Query	No	Name or code substring.
category	Query	No	Category filter or all.

Sample Request Body

```
None (Query Parameters)
```

Sample JSON Response

```
{
  "success": true,
  "tests": [{
    "id": "68f1t...",
    "code": "CBC",
    "name": "Complete Blood Count",
    "category": "Hematology",
    "price": 450
  }]
}
```

GET **/api/diagnostics/packages**

Access: Authenticated User

Description: Bundled executive / diabetic packages with discount metadata.

Parameter	Type	Required	Description
Authorization	Header	Yes	Session cookie.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "packages": [{
    "id": "pkg-executive",
    "name": "Executive Health Check",
    "discountPercent": 15,
    "testCodes": ["CBC", "LIPID", "LFT", "FBS", "HBA1C"]
  }]
}
```

POST /api/diagnostics/orders

Access: Authenticated Patient

Description: Creates a home collection order. If a phlebotomist exists, status becomes assigned with etaMinutes.

Parameter	Type	Required	Description
testIds	Array	Conditional	DiagnosticTest ObjectIds (required if no packageId).
packageId	String	Conditional	pkg-executive or pkg-diabetic.
address	String	Yes	Residential collection address.
contactPhone	String	Yes	Contact mobile number.
preferredDate	String	Yes	Collection date YYYY-MM-DD.
timeSlot	String	Yes	e.g. 08:00-10:00.

Sample Request Body

```
{
  "testIds": ["68f1t-cbc"],
  "address": "House 14, Road 5, Dhanmondi",
  "contactPhone": "01712345678",
  "preferredDate": "2026-10-10",
  "timeSlot": "08:00-10:00"
}
```

Sample JSON Response (201)

```
{
  "success": true,
  "order": {
    "id": "68f1ord...",
    "status": "pending_dispatch",
    "totalAmount": 450,
    "preferredDate": "2026-10-10"
  }
}
```

GET /api/diagnostics/orders/me

Access: Authenticated Patient

Description: Lists the patient's diagnostic orders (newest first).

Parameter	Type	Required	Description
Authorization	Header	Yes	Patient session.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "orders": [{ "id": "68f1ord...", "status": "assigned", "totalAmount"
}
```

GET /api/diagnostics/orders/assigned

Access: Phlebotomist, Administrator

Description: Dispatch queue of incomplete orders assigned to the rider (admins see all open orders).

Parameter	Type	Required	Description
Authorization	Header	Yes	Phlebotomist or admin session.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "orders": [{ "id": "68f1ord...", "address": "House 14, Road 5", "etal
}
```

GET /api/diagnostics/orders/:id

Access: Owner Patient, Assigned Staff, Shared Doctor, Administrator

Description: Order detail. Patients also receive downloadToken (HMAC) for PDF retrieval.

Parameter	Type	Required	Description
id	Path	Yes	DiagnosticOrder ObjectId.

Sample Request Body

```
None (GET Request)
```

Sample JSON Response

```
{
  "success": true,
  "order": {
    "id": "68f1ord...",
    "status": "completed",
    "downloadToken": "eyJhbGciOi..."
  }
}
```

PATCH /api/diagnostics/orders/:id/status

Access: Assigned Phlebotomist, Administrator

Description: Updates collection workflow: assigned, en_route, collected, at_lab, completed, cancelled.

Parameter	Type	Required	Description
id	Path	Yes	DiagnosticOrder ObjectId.
status	Enum	No	Valid ORDER_STATUSES value.
barcodeId	String	No	Specimen barcode.
etaMinutes	Integer	No	Updated rider ETA.
collectionNote	String	No	Field note.
criticalFlags	Array	No	Critical result flags.

Sample Request Body

```
{
  "status": "collected",
  "barcodeId": "BC-771-A"
}
```

Sample JSON Response

```
{
  "success": true,
  "order": { "id": "68f1ord...", "status": "collected", "barcodeId": "
}
```

POST /api/diagnostics/orders/:id/report

Access: System Administrator

Description: Uploads a finalized PDF (multipart field report, max 8 MB). Stores SHA-256 on the order and marks completed.

Parameter	Type	Required	Description
id	Path	Yes	DiagnosticOrder ObjectId.
report	File	Yes	PDF multipart binary (field name report).

Sample Request Body

```
Multipart form-data
(report=report_771.pdf)
```

Sample JSON Response

```
{
  "success": true,
  "reportSha256": "e3b0c44298fc1c14...",
  "order": { "id": "68f1ord...", "status": "completed" }
}
```

GET /api/diagnostics/orders/:id/download

Access: Owner Patient (HMAC token), Shared Doctor, Administrator

Description: Streams the stored PDF. Patients must pass ?token= from GET order. Response Content-Type: application/pdf; header X-Report-SHA256.

Parameter	Type	Required	Description
id	Path	Yes	DiagnosticOrder ObjectId.
token	Query	Conditional	HMAC download token (required for owning patient).

Sample Request Body

```
None (GET Request)
Produces application/pdf
```

Sample JSON Response

```
Binary PDF stream
Header: X-Report-SHA256: e3b0c4...
```

POST /api/diagnostics/orders/:id/share-doctor

Access: Owner Patient

Description: Grants a doctor read access to the completed report.

Parameter	Type	Required	Description
id	Path	Yes	DiagnosticOrder ObjectId.
doctorId	String	Yes	Doctor User ObjectId.

Sample Request Body

```
{
  "doctorId": "68f1doc..."
}
```

Sample JSON Response

```
{
  "success": true,
  "order": { "id": "68f1ord...", "sharedDoctorIds": ["68f1doc..."] }
}
```

6.6 Module 6: Administration & Moderation

GET /api/admin/doctors

Access: System Administrator

Description: Lists doctors for BMDC moderation with counts by approval status.

Parameter	Type	Required	Description
status	Query	No	pending (default), approved, rejected, all.

Sample Request Body

```
None (Query Parameters)
```

Sample JSON Response

```
{
  "success": true,
  "counts": { "pending": 4, "approved": 12, "rejected": 1 },
  "doctors": [{
    "id": "68f1doc...",
    "fullName": "Dr. Nusrat Jahan",
    "doctorApprovalStatus": "pending"
  }]
}
```

PATCH /api/admin/doctors/:id/approval

Access: System Administrator

Description: Approves or rejects a doctor. Approved doctors appear in GET /api/doctors and may create chambers.

Parameter	Type	Required	Description
id	Path	Yes	Doctor User ObjectId.
status	Enum	Yes	approved, rejected, or pending.

Sample Request Body

```
{
  "status": "approved"
}
```

Sample JSON Response

```
{
  "success": true,
  "doctor": {
    "id": "68f1doc...",
    "fullName": "Dr. Nusrat Jahan",
    "doctorApprovalStatus": "approved"
  }
}
```

6.7 Notifications, Family Care, Analytics and Administration Extensions

These endpoints complete the SRS functional requirements beyond the core booking, queue and diagnostics flows. All list endpoints accept ?page= and ?limit= and return a pagination object { page, limit, total, pages }. Every state-changing admin action is written to the audit log.

Method	Path	Access	Purpose (SRS)
GET	/api/notifications	Any signed-in user	Paginated in-app notifications + unread count (FR-05, FR-13)
PATCH	/api/notifications/:id/read	Owner	Mark one notification read
POST	/api/notifications/read-all	Any signed-in user	Mark all notifications read
GET	/api/family	Patient	List family members (max 10)
POST	/api/family	Patient	Add member: fullName, relation, gender, age, phone, bloodGroup
PATCH	/api/family/:id	Owner patient	Edit a member
DELETE	/api/family/:id	Owner patient	Remove a member
POST	/api/schedules/slots/book	Patient	Now accepts familyMemberId to book for a family member
GET	/api/appointments/doctor/summary?date=	Doctor, Admin	End-of-day summary: booked, completed, no-show, walk-ins, fees (FR-15)
GET	/api/diagnostics/trends?code=	Patient	Per-test result history for charts (FR-27)
POST	/api/diagnostics/orders/:id/report	Admin	Multipart: report (PDF) + results JSON; derives abnormal flags (FR-21, FR-24)
GET	/api/diagnostics/orders	Admin	All orders, paginated, ?status= filter
POST	/api/diagnostics/tests	Admin	Create a catalogue test
PATCH	/api/diagnostics/tests/:id	Admin	Update price, details or retire a test
GET	/api/admin/audit	Admin	Paginated audit log, ?action= filter
GET	/api/ready	Public	Readiness probe (database connectivity)
POST	/api/auth/register (multipart)	Public	Doctor signup with BMDC number and verification documents (PDF/JPG/PNG, 5 MB)
POST	/api/users/me/documents	Doctor	Add or replace verification documents; a rejected doctor returns to pending
GET	/api/users/me/documents/:docId	Owner doctor	Open own uploaded document
GET	/api/admin/doctors/:id	Admin	Full doctor details, document checklist and file links before approval
GET	/api/admin/doctors/:id/documents/:docId	Admin	Open an uploaded document inline
PATCH	/api/admin/doctors/:id/approval	Admin	Approve (needs all documents) or reject (reason required, sent to the doctor)
PATCH	/api/appointments/:id/status	Doctor, Admin	status=cancelled needs a reason; patient is notified with the next available slot
POST	/api/appointments/:id/cancel	Patient, Admin	Cancel with optional reason; slot reopens and the doctor is notified

POST	/api/schedules/:id/cancel-day	Doctor	Close one chamber day: cancels and notifies every booked patient
DELETE	/api/schedules/:id/closed-dates/:date	Doctor	Reopen a cancelled day
GET	/api/diagnostics/centers	Any signed-in user	Diagnostic centers with quote for ?packageId= / ?testIds= / ?codes= (price, status, hours)
GET	/api/diagnostics/centers/:id	Any signed-in user	Center detail with its price list
POST	/api/diagnostics/centers	Admin	Create a center (starts with catalogue prices)
PATCH	/api/diagnostics/centers/:id	Admin	Update hours, fees, prices or pause orders
POST	/api/diagnostics/orders	Patient	Order now accepts centerId; prices, fee and status timeline come from that center

Platform hardening: helmet security headers, request logging (morgan), rate limiting (300 requests per minute per IP; 30 failed attempts per 15 minutes on sign-in, register and password endpoints), JSON-only error envelope, SMS gateway abstraction (SMS_PROVIDER=console|http), Docker images, docker-compose and a GitHub Actions CI pipeline.

6.8 Test Execution and Results

Every endpoint above is verified by automated HTTP integration tests (Node.js node:test + Supertest) against the real Express app and an in-memory MongoDB (mongodb-memory-server). Two kinds of test are kept: Pass tests (PT-xx) send valid requests and assert the success contract; Fail tests (FT-xx) send invalid, unauthorised or conflicting requests and assert the documented 4xx rejection with { success: false }.

Item	Value
Command	cd backend && npm test
Runner	node --test --test-concurrency=1 test/**/*.test.js
Database	MongoDB Memory Server (isolated; no Atlas required)
Pass tests (PT)	66 executed / 66 passed (grouped into the 57 scenarios of Section 6.9)
Fail tests (FT)	96 executed / 96 passed (each rejected with the expected 4xx)
Total	162 executed / 162 passed / 0 failed

Defect found by fail testing: FT-QUEUE-03 first returned 500 Internal Server Error for an unknown queue session and malformed ids. The queue service now raises 404 Not Found, and the error handler maps malformed ObjectIds (CastError) to 400 Bad Request.

6.9 Pass Test Cases (valid requests)

ID	Test case	Endpoint	Expected result
PT-01	Register creates account and session	POST /api/auth/register	201 + Set-Cookie
PT-02	Login with valid credentials	POST /api/auth/login	200 + cookie
PT-03	Logout clears the session	POST /api/auth/logout	200; GET /me then 401
PT-04	Forgot and reset password	POST /api/auth/forgot-password, /reset-password	200; new password works
PT-05	Admin allowed on overview route	GET /api/users/admin/overview	200
PT-06	Profile read and update	GET/PATCH /api/users/me	200; name updated
PT-07	Health probe	GET /api/health	200 status ok
PT-08	Approved doctor creates chamber + schedule	POST /api/chambers, /api/schedules	201
PT-09	Doctor updates schedule times	PATCH /api/schedules/:id	200 startTime 10:30
PT-10	Doctor deletes availability	DELETE /api/schedules/:id	200; list empty
PT-11	Public doctor schedules	GET /api/schedules/doctors/:id	200 length 1

PT-12	Inactive schedule hidden from public	PATCH /api/schedules/:id/status	public list length 0
PT-13	Patient books slot, serial #1 assigned	POST /api/schedules/slots/book	201 serialNumber 1
PT-14	Slot quota respected	POST /api/schedules/slots/book	201 then quota enforced
PT-15	Appointment history	GET /api/appointments/me	200 length 1
PT-16	Patient cancels booking	POST /api/appointments/:id/cancel	200 status cancelled
PT-17	Doctor profile with chambers	GET /api/doctors/:id	200 fullName
PT-18	Directory lists approved doctors	GET /api/doctors	200 doctors[0]
PT-19	Admin approval reveals doctor	PATCH /api/admin/doctors/:id/approval	list 0 then 1
PT-20	Admin lists and rejects doctors	GET/PATCH /api/admin/doctors	200 rejected
PT-21	Queue session ensure + advance	POST /api/queue/sessions, /advance	200 nowServing set
PT-22	Patient live queue today	GET /api/queue/me/today	200 serialNumber
PT-23	Assistant inserts walk-in #nB	POST /api/queue/sessions/:id/walk-in	201
PT-24	Diagnostic catalogue	GET /api/diagnostics/tests	200 length >= 3
PT-25	Create home collection order	POST /api/diagnostics/orders	201 pending_dispatch
PT-26	List my orders	GET /api/diagnostics/orders/me	200 length 1
PT-27	Packages catalogue	GET /api/diagnostics/packages	200 length >= 2
PT-28	Appointment QR SmartPass	GET /api/appointments/:id/qr	200 data:image/png
PT-29	Booking creates in-app notification + SMS	POST /api/schedules/slots/book	201; notification + SMS logged
PT-30	Mark one / all notifications read	PATCH /api/notifications/:id/read, POST /read-all	200; unread 0
PT-31	Family CRUD and booking for a member	/api/family, POST /slots/book	201; bookedForName set
PT-32	Doctor end-of-day summary	GET /api/appointments/doctor/summary	200 counts and fees
PT-33	Paginated appointment list	GET /api/appointments/me?limit=1	200 pagination.total 1
PT-34	Report upload with results, flags, trends	POST .../report, GET /api/diagnostics/trends	200 flagged high; series returned
PT-35	Admin creates and updates a catalogue test	POST/PATCH /api/diagnostics/tests	201 / 200; audit entry
PT-36	Admin lists all orders (paginated)	GET /api/diagnostics/orders	200 pagination
PT-37	Patient shares report, doctor notified	POST .../share-doctor	200; doctor notification
PT-38	Readiness probe and security headers	GET /api/ready	200; nosniff; no x-powered-by
PT-39	localhost and 127.0.0.1 origins allowed (CORS)	any /api route	200 with credentials header
PT-40	Doctor signup stores BMDC details and documents on disk	POST /api/auth/register	201; 3 documents; files in project folder
PT-41	Admin reads full doctor details and document checklist	GET /api/admin/doctors/:id	200 documents + requiredDocuments
PT-42	Admin opens an uploaded document inline	GET /api/admin/doctors/:id/documents/:docId	200 correct content type, nosniff
PT-43	Doctor views own file and re-uploads after rejection	POST /api/users/me/documents	200; status back to pending
PT-44	Rejection reason stored and sent to the doctor	PATCH /api/admin/doctors/:id/approval	200; note + notification
PT-45	Patient cancels: slot reopens, doctor told	POST /api/appointments/:id/cancel	200 cancelledBy patient

PT-46	Doctor cancels an appointment: reason, SMS, next slot	PATCH /api/appointments/:id/status	200; patient notified with next slot
PT-47	Doctor cancels a whole chamber day, then reopens it	POST /api/schedules/:id/cancel-day	200 all patients notified; date closed
PT-48	Removing a schedule cancels future bookings	DELETE /api/schedules/:id	200 cancelledBookings 1
PT-49	Pausing a schedule cancels future bookings	PATCH /api/schedules/:id/status	200 cancelledBookings 1
PT-50	Changing times cancels only bookings that no longer fit	PATCH /api/schedules/:id	200 cancelledBookings 1
PT-51	Compare centers: quote, cheapest, recommended	GET /api/diagnostics/centers	200 sorted with quotes
PT-52	Order at chosen center uses its prices	POST /api/diagnostics/orders	201 center + itemised total
PT-53	Free collection above threshold; package discount	POST /api/diagnostics/orders	201 fee 0, discount applied
PT-54	No choice: cheapest open eligible center assigned	POST /api/diagnostics/orders	201 auto-assigned
PT-55	Status timeline grows with every change	PATCH .../status, POST .../report	4 history entries
PT-56	Admin edits center; audit entry written	PATCH /api/diagnostics/centers/:id	200; audit center.update
PT-57	New center starts with catalogue prices	POST /api/diagnostics/centers	201 prices copied

6.10 Fail Test Cases (invalid, unauthorised or conflicting requests)

ID	Fail scenario	Endpoint	Expected
FT-AUTH-01	Register with an already used email	POST /api/auth/register	409
FT-AUTH-02	Register with malformed email	POST /api/auth/register	400
FT-AUTH-03	Register without full name	POST /api/auth/register	400
FT-AUTH-04	Login with wrong password	POST /api/auth/login	401
FT-AUTH-05	Login with unknown account	POST /api/auth/login	401
FT-AUTH-06	Read /me without session cookie	GET /api/auth/me	401
FT-AUTH-07	Read /me with tampered JWT	GET /api/auth/me	401
FT-AUTH-08	Reset password with invalid token	POST /api/auth/reset-password	400
FT-AUTH-09	Change password with wrong current password	POST /api/auth/change-password	400 / 401
FT-USER-01	Read profile without session	GET /api/users/me	401
FT-DOC-01	Doctor directory without session	GET /api/doctors	401
FT-CHAM-01	Patient creates a chamber	POST /api/chambers	403
FT-CHAM-02	Unapproved doctor creates a chamber	POST /api/chambers	
FT-SCHED-01	Schedule end time before start time	POST /api/schedules	400
FT-SCHED-02	Patient publishes a schedule	POST /api/schedules	403
FT-SCHED-03	Overlapping schedule, same chamber/day	POST /api/schedules	409
FT-SCHED-04	Schedule for an unknown chamber	POST /api/schedules	400 / 404
FT-BOOK-01	Book a slot without session	POST /api/schedules/slots/book	401

FT-BOOK-02	Book with missing fields	POST /api/schedules/slots/book	400
FT-BOOK-03	Book an already taken slot	POST /api/schedules/slots/book	409
FT-BOOK-04	Doctor tries to book a patient slot	POST /api/schedules/slots/book	403
FT-APPT-01	Other patient reads my appointment	GET /api/appointments/:id	403 / 404
FT-APPT-02	Patient changes appointment status	PATCH /api/appointments/:id/status	403
FT-APPT-03	Doctor calls patient booking history	GET /api/appointments/me	403
FT-QUEUE-01	Advance queue without session	POST /api/queue/sessions/:id/advance	401
FT-QUEUE-02	Patient advances the queue	POST /api/queue/sessions/:id/advance	403
FT-QUEUE-03	Assistant advances an unknown session	POST /api/queue/sessions/:id/advance	404
FT-QUEUE-04	Doctor inserts an emergency walk-in	POST /api/queue/sessions/:id/walk-in	403
FT-QUEUE-05	Malformed session id	POST /api/queue/sessions/:id/advance	400
FT-DIAG-01	Doctor orders a home collection	POST /api/diagnostics/orders	403
FT-DIAG-02	Order without address/phone/date/slot	POST /api/diagnostics/orders	400
FT-DIAG-03	Order without any test selected	POST /api/diagnostics/orders	400
FT-DIAG-04	Order with unknown package	POST /api/diagnostics/orders	400
FT-DIAG-05	Other patient reads / downloads my report	GET /api/diagnostics/orders/:id[/download]	403 / 404
FT-DIAG-06	Patient uploads a lab report	POST /api/diagnostics/orders/:id/report	403
FT-DIAG-07	Patient updates collection status	PATCH /api/diagnostics/orders/:id/status	403
FT-ADMIN-01	Patient lists doctors for moderation	GET /api/admin/doctors	403
FT-ADMIN-02	Approval with invalid status value	PATCH /api/admin/doctors/:id/approval	400
FT-ADMIN-03	Approval of an unknown doctor	PATCH /api/admin/doctors/:id/approval	404
FT-ADMIN-04	Patient reads the audit log	GET /api/admin/audit	403
FT-ADMIN-05	Orders list with invalid status filter	GET /api/diagnostics/orders	400
FT-NOTIF-01	Notifications without session	GET /api/notifications	401
FT-NOTIF-02	Mark read with malformed id	PATCH /api/notifications/:id/read	400
FT-NOTIF-03	Mark another user's notification	PATCH /api/notifications/:id/read	404
FT-FAM-01	Doctor uses family care	GET /api/family	403
FT-FAM-02	Invalid relation	POST /api/family	400
FT-FAM-03	Edit another user's family member	PATCH /api/family/:id	404
FT-FAM-04	Book for someone else's family member	POST /api/schedules/slots/book	404
FT-FAM-05	More than 10 family members	POST /api/family	409
FT-SUM-01	Patient reads doctor summary	GET /api/appointments/doctor/summary	403
FT-SUM-02	Summary without a valid date	GET /api/appointments/doctor/summary	400
FT-DIAG-08	Non-PDF report upload	POST /api/diagnostics/orders/:id/report	400
FT-DIAG-09	Results that are not valid JSON	POST /api/diagnostics/orders/:id/report	400
FT-DIAG-10	Patient creates a catalogue test	POST /api/diagnostics/tests	403
FT-DIAG-11	Duplicate catalogue code	POST /api/diagnostics/tests	409
FT-DIAG-12	Negative price	POST /api/diagnostics/tests	400
FT-DIAG-13	Doctor reads patient trends	GET /api/diagnostics/trends	403

FT-DIAG-14	Update an unknown catalogue test	PATCH /api/diagnostics/tests/:id	404
FT-DIAG-15	Share report with an unknown doctor	POST .../share-doctor	404
FT-RATE-01	Request flood beyond the limiter	any /api route	429
FT-CORS-01	Request from an unknown origin	any /api route	no CORS headers
FT-DOC-02	Doctor signup without documents	POST /api/auth/register	400; no account created
FT-DOC-03	Signup missing one required document	POST /api/auth/register	400 missingDocuments
FT-DOC-04	Unsupported document type	POST /api/auth/register	400
FT-DOC-05	File content does not match its type	POST /api/auth/register	400
FT-DOC-06	Signup without BMDC number	POST /api/auth/register	400
FT-DOC-07	Document larger than 5 MB	POST /api/auth/register	413
FT-DOC-08	Patient reads doctor details / documents	GET /api/admin/doctors/:id[/documents/:id]	403
FT-DOC-09	Doctor opens another doctor's documents	GET /api/admin/doctors/:id/documents/:id	403 / 404
FT-DOC-10	Unknown doctor or malformed ids	GET /api/admin/doctors/:id	404 / 400
FT-DOC-11	Patient uploads doctor documents	POST /api/users/me/documents	403
FT-ADMIN-06	Reject a doctor without a reason	PATCH /api/admin/doctors/:id/approval	400
FT-ADMIN-07	Approve a doctor with no documents	PATCH /api/admin/doctors/:id/approval	
FT-CANCEL-01	Doctor cancels without a reason	PATCH /api/appointments/:id/status	400
FT-CANCEL-02	Cancel someone else's appointment	POST /api/appointments/:id/cancel	403
FT-CANCEL-03	Cancel chamber day without a reason	POST /api/schedules/:id/cancel-day	400
FT-CANCEL-04	Cancel a day that has passed	POST /api/schedules/:id/cancel-day	400
FT-CANCEL-05	Cancel a day the schedule does not run	POST /api/schedules/:id/cancel-day	400
FT-CANCEL-06	Cancel the same day twice	POST /api/schedules/:id/cancel-day	409
FT-CANCEL-07	Patient cancels a chamber day	POST /api/schedules/:id/cancel-day	403
FT-CANCEL-08	Another doctor cancels my chamber day	POST /api/schedules/:id/cancel-day	404
FT-CANCEL-09	Reopen a day that was not cancelled	DELETE /api/schedules/:id/closed-dates/:date	404
FT-CANCEL-10	Cancel an already cancelled appointment	POST /api/appointments/:id/cancel	400
FT-CENTER-01	Patient creates or edits a center	POST/PATCH /api/diagnostics/centers	403
FT-CENTER-02	Order with malformed center id	POST /api/diagnostics/orders	400
FT-CENTER-03	Order at an unknown center	POST /api/diagnostics/orders	404
FT-CENTER-04	Center lacks a selected test	POST /api/diagnostics/orders	
FT-CENTER-05	Center not accepting orders	POST /api/diagnostics/orders	
FT-CENTER-06	Center without home collection	POST /api/diagnostics/orders	400
FT-CENTER-07	Unknown package when listing centers	GET /api/diagnostics/centers	400
FT-CENTER-08	Center without name or with bad hours	POST /api/diagnostics/centers	400
FT-CENTER-09	Duplicate center name	POST /api/diagnostics/centers	409
FT-CENTER-10	Update or read an unknown center	PATCH/GET /api/diagnostics/centers/:id	404 / 400

FT-CENTER-11	No center can serve every test	POST /api/diagnostics/orders	
FT-CENTER-12	List centers without a session	GET /api/diagnostics/centers	401
FT-CENTER-13	Negative price on a center	POST /api/diagnostics/centers	400

6.11 Requirements Traceability and Techniques

SRS use case	Primary endpoints	Pass tests	Fail tests
UC-01 Register / login	/api/auth/*	PT-01 ... PT-04	FT-AUTH-01 ... 09
UC-02 Profile	/api/users/me	PT-06	FT-USER-01
UC-05 Doctor availability	/api/chambers, /api/schedules	PT-08 ... PT-12	FT-CHAM-01/02, FT-SCHED-01 ... 04
UC-08 Book serial	/api/schedules/slots/book	PT-13, PT-14	FT-BOOK-01 ... 04
UC-09 Bookings / cancel / QR	/api/appointments/*	PT-15, PT-16, PT-28	FT-APPT-01 ... 03
UC-10 Live queue	/api/queue/*, Socket.io	PT-21 ... PT-23	FT-QUEUE-01 ... 05
UC-12 Diagnostics	/api/diagnostics/*	PT-24 ... PT-27, PT-51 ... PT-57	FT-DIAG-01 ... 07, FT-CENTER-01 ... 13
FR-28 Doctor documents and review	/api/auth/register, /api/admin/doctors/:id	PT-40 ... PT-44	FT-DOC-02 ... 11, FT-ADMIN-06, 07
UC-09 Cancellation handling	/api/appointments, /api/schedules	PT-45 ... PT-50	FT-CANCEL-01 ... 10
UC-15 BMDC moderation, audit	/api/admin/*	PT-19, PT-20, PT-35	FT-ADMIN-01 ... 05
FR-05 / FR-13 Notifications	/api/notifications	PT-29, PT-30	FT-NOTIF-01 ... 03
Family care	/api/family	PT-31	FT-FAM-01 ... 05
FR-15 Day summary	/api/appointments/doctor/summary	PT-32	FT-SUM-01, 02
FR-21 / 23 / 24 / 27 Reports	/api/diagnostics/*	PT-34 ... PT-37	FT-DIAG-08 ... 15
Platform hardening	/api/ready, rate limits	PT-38	FT-RATE-01

Techniques: black-box specification-based testing; equivalence partitioning (valid vs invalid credentials, approved vs pending doctors, each role vs forbidden roles); boundary cases (password length, end time before start time, duplicate slot); state-based tests (register, login, logout; book, cancel). Not yet automated: Socket.io latency (< 150 ms), PDF upload / HMAC download, phlebotomist dispatch workflow.